

Incorporating an ONNX model into AVEVA Process Simulation

Note: Example below is to help you integrate your ONNX model into AVEVA Process Simulation. The inputs and outputs are different than the model you have trained for the competition.

Imagine you are trying to model the behavior of a pump in your process but do not have access to the performance curve. In this example, we will show how you can use an ML model in AVEVA Process Simulation to predict the **head** and **efficiency** of the pump.

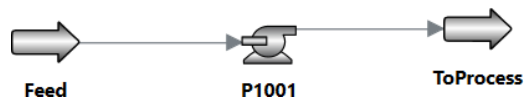


Figure 4 - A simple pump model in AVEVA Process Simulation

Step 1: Determine inputs and outputs

In this example, we use **volumetric flow rate (P1001.Q)** and **motor speed (P1001.Speed)** as the input variables for the ML model. These values are easy to measure in the process and describe the behavior of a pump well (to convince yourself of this, look at a standard performance curve for a pump).

We have chosen **discharge pressure (P1001.P2)** and **power (P1001.Power)** as output variables. These are easy to measure in the process and will be used to calculate the corresponding **head** and **efficiency** in AVEVA Process Simulation.

Step 2: Train your ML model

In this example, we have chosen a k-nearest neighbors regression model from scikit-learn to estimate the discharge pressure and power. A single line of the code is provided below to demonstrate the output of the model. Remember this is only an example.

```
KNNmodel = KNeighborsRegressor(n_neighbors=2).fit(X_train, y_train)
```

Step 3: Export as an ONNX model

After training the ML model, we need to export it to the ONNX format for use within AVEVA Process Simulation. The Python code below converts the KNNmodel from scikit-learn to an equivalent model in ONNX format.

NOTE: we recommend converting your dataset to a **float32** variable type for compatibility with AVEVA Process Simulation.

```
from skl2onnx import to_onnx
onnx = to_onnx(KNNmodel, X_train.to_numpy(dtype="float32"))

with open("PumpModelONNX.onnx", "wb") as f:
    f.write(onnx.SerializeToString())
```

Step 4: Import the Preview library in AVEVA Process Simulation

The Preview library of AVEVA Process Simulation includes the **ONNXAdaptor** model that we will use to call our trained ML model. To import the Preview library, first check that you are using the **Model Writing** role. Then, click the [+] button in the bottom left corner of the screen (under the list of libraries), select **Import Library**, and open the **Preview** library (...\\SimCentral Examples\\Libraries\\Preview.libx).

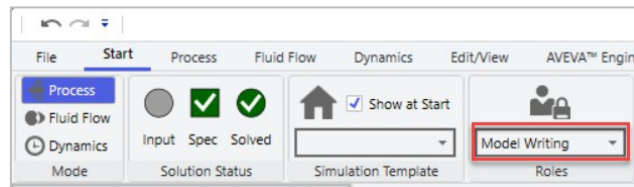


Figure 5 - Model Writing role in AVEVA Process Simulation

Step 5: Configure the ONNX adaptor

To start, we drag/drop the **ONNXAdaptor** model into the simulation to create model **ONNX1**.

From the Mini Inspector, update the **Path** parameter to direct the adapter to the ONNX model you created in Step 3 above. To populate the **Description** field, change the **Setup** parameter to **Solve** then change back to **Configure**. Copy the information from the **Description** parameter to **NI**, **NO**, **InputLabel**, and **OutputLabel** as shown below.

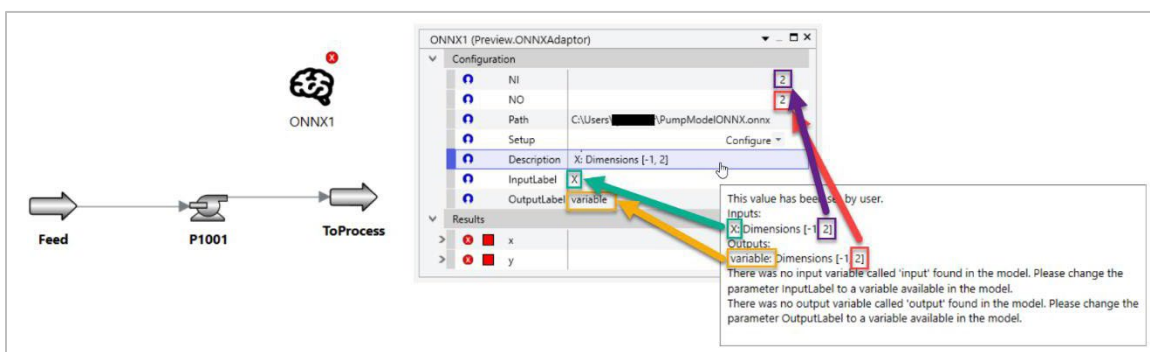


Figure 6 - Initial configuration of ONNX1 model

Step 6: Connect the ONNX model inputs to variables in the simulation

Next, we will indicate which variables in the simulation correspond to the inputs of the ML model. In this example, we will use flowsheet equations. The model help file for the **ONNXAdaptor** explains how you can embed this model within another model if desired.

Recall that we identified **P1001.Speed** and **P1001.Q** as inputs in Step 1. We add simple flowsheet equations setting these equal to the ONNX input variables:

$$\text{Eqn1: } \text{ONNX1.x}[1] = \text{P1001.Speed}$$

$$\text{Eqn2: } \text{ONNX1.x}[2] = \text{P1001.Q} * 3600$$

NOTE: be careful of units! Double check that the values displayed in the **ONNXAdaptor** model match the values and units of measure you used to train your model. Equations in AVEVA Process Simulation use the internal, default UOMs which may be different than the ones you have used to train your model.

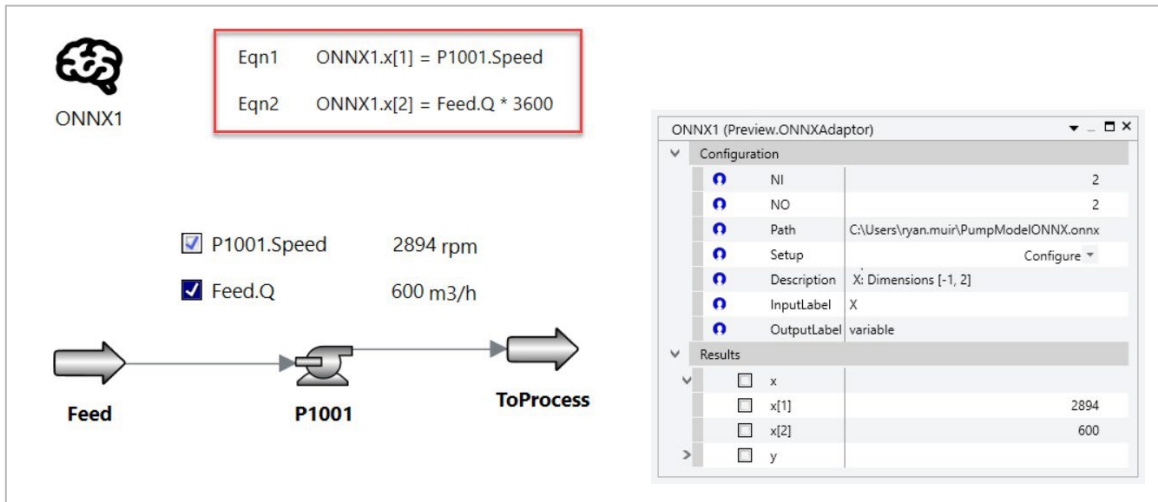


Figure 7 - Flowsheet equations defining the inputs to the ONNX model

Step 7: Confirm the ONNX model is working as expected

After configuring the model inputs, change the **Setup** parameter to **Solve**. Check that the ONNX model is producing outputs in the **Results** group. If not, check your log file for more information (...\\SimCentral\\Logs\\Server.log).

Step 8: Map the outputs to variables in the simulation

After the ONNX model has been solved, we can map the output values to their corresponding variables in the simulation. In this example, we will use flowsheet equations to complete the mapping.

Earlier, we identified the outputs of the ML model as **P1001.P2** and **P1001.Power**, so we add equations mapping these variables to the outputs of the ONNX model:

$$\text{Eqn3: } \text{P1001.P2} = \text{ONNX1.y}[1]$$

$$\text{Eqn4: } \text{P1001.Power} = \text{ONNX1.y}[2]$$

After making these specifications, the model is overspecified by two variables. Therefore, we must choose two variables to unspecify. **P1001.P2** and **P1001.Power** would be the obvious choices, but neither is specified in the model by default. Instead, we unspecify **P1001.DP** and **P1001.eta** to re-square the model. These are in the same equation sets as **P1001.P2** and **P1001.Power**, respectively. The model is now complete.

Understanding which variables to unspecify is an important part of configuring your ONNX model. Your selections (and the corresponding justification of those selections) will be used when grading your responses for the competition.

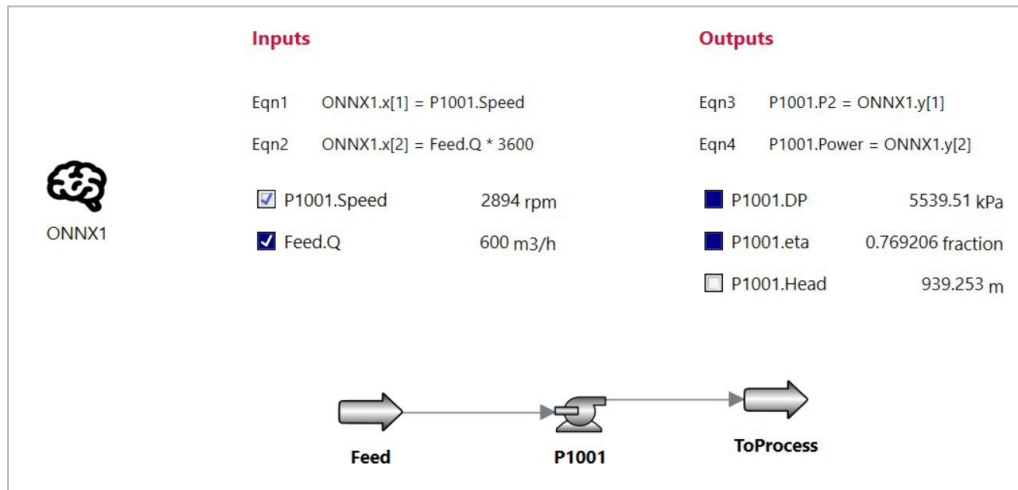


Figure 8 - Final ONNX model to estimate pump performance from input data